

LLM Fundamentals

Tokens, context windows, temperature and the two phases behind every response. The vocabulary you are expected to already know.

How an LLM Works

CORE

A language model does exactly one thing. Given a sequence of tokens, it produces a probability for every token in its vocabulary, one of them is picked, appended to the sequence, and the whole thing runs again.

THE LOOP, ONCE PER TOKEN



WHAT THAT ACTUALLY MEANS



It samples, it does not look up



Same prompt, different output



No memory between calls



Everything is billed per token

The model runs a full forward pass for every single token it writes. That one fact explains most of the cost and latency you will ever debug.

Tokens

THE UNIT

Models never see characters or words. Text is split into tokens, subword pieces from a fixed vocabulary, and each token becomes an integer id. Everything downstream is counted in these units.

EXAMPLE: "TOKENIZATION IS UNBELIEVABLE"

Token

ization

is

un

bel

iev

able

Three words, seven tokens. Common words are one token, rare words shatter into pieces, and the leading space usually belongs to the token.

RULES OF THUMB FOR ENGLISH

4

characters per token,
roughly

75

words per 100 tokens

2x

more tokens for many
other languages

WHERE TOKEN COUNTS QUIETLY EXPLODE



Raw HTML and base64 blobs



UUIDs, hashes, long ids

Code, JSON, long numbers and non English text all tokenize badly. The same meaning can cost you twice the tokens depending on how you format it.

One Kit to Clear Your AI Interview

KIT

After seeing how AI interviews are evolving, I created an **AI Interview Kit**. It covers the areas companies actually test:



AI fundamentals



AI agents



Transformers in depth



Memory systems



LLM architectures



Databases



Prompt engineering



Backend for AI systems



Fine tuning



Deployment strategies



RAG systems



GenAI system design

Everything structured to help you prepare.

↓ [Check the caption to enroll](#)

Count Tokens, Not Words

COST

Never estimate by eye. Count tokens with the tokenizer of the model you are actually calling, because every provider splits text differently.

```
import tiktoken

enc = tiktoken.get_encoding("o200k_base")
ids = enc.encode("Tokenization is unbelievable")

print(len(ids))                # 7 tokens
print([enc.decode([i]) for i in ids])

# cost of one call, per million token pricing
cost = (in_tok / 1_000_000) * in_price \
      + (out_tok / 1_000_000) * out_price
```

A WORKED EXAMPLE

12k

input tokens at 3 dollars
per million

800

output tokens at 15
dollars per million

4.8c

per call, so 480 dollars at
10k calls a day

Output tokens usually cost three to five times more than input tokens. Capping answer length is often the fastest cost win you have.

The Context Window

BUDGET

The context window is the total token budget for one call. Input and output share it. Nothing outside it exists for the model, and the moment you exceed it you get a truncation or an error.

A 128K WINDOW, SPENT



WHAT BREAKS IN PRODUCTION



Silent truncation of history



No room left for the answer



Recall drops in the middle



Latency and cost climb

HOW TO STAY INSIDE IT



Budget every section



Reserve output space first

A big window is a limit, not a plan. Decide what deserves to be in context before you decide how much fits.

Input Window vs Max Output

LIMITS

Two different numbers that people constantly confuse. The context window is the whole call. Max output tokens is a separate, much smaller ceiling on what the model is allowed to generate.

CONTEXT WINDOW

- Input plus output combined
- Fixed by the model
- Often 128k to 1M tokens
- Exceeding it is a hard error

MAX OUTPUT TOKENS

- Generation ceiling only
- You set it per request
- Often 4k to 64k tokens
- Exceeding it truncates mid sentence

```
resp = client.messages.create(  
    model="claude-opus-4-5",  
    max_tokens=1024,          # output ceiling only  
    messages=[{"role": "user", "content": prompt}],  
)
```

```
usage = resp.usage           # input_tokens, output_tokens  
print(resp.stop_reason)     # end_turn or max_tokens
```

Usable output equals the smaller of `max_tokens` and whatever the window has left. Fill 190k of a 200k window and no `max_tokens` setting will save you.

Phase One: Prefill

READING

Before a single token comes out, the model reads your whole prompt in one parallel pass and builds the internal key and value vectors for every input token.

All Input
Tokens

,

One Parallel
Pass

,

KV Cache
Filled

,

First Token
Emitted

CHARACTERISTICS



Parallel across all input tokens, so it saturates the GPU



Compute bound, the phase that scales with prompt length



Its duration is what you measure as time to first token



Repeated prefixes can be cached and skipped on later calls

A slow first token is almost always a prefill problem. The fix is a shorter or cached prompt, never a lower temperature.

Phase Two: Decoding

WRITING

Now the model generates one token at a time. Each token is appended to the sequence and fed back in, so token 500 cannot start until token 499 exists.

Last Token
In

,

Attend to
KV Cache

,

Sample
Next Token

,

Append and
Repeat

CHARACTERISTICS



Strictly sequential, one forward pass per output token



Memory bandwidth bound, not compute bound like prefill



Measured as time per output token, then tokens per second

THE ARITHMETIC THAT MATTERS

0.4s

prefill of a 12k prompt, your time to first token

12.5s

500 output tokens at 25 ms each

Long prompts hurt first token latency. Long answers hurt total latency. Two problems, two different fixes.

The KV Cache

MEMORY

Without a cache the model would recompute attention over the entire sequence for every new token. The KV cache stores those vectors once and reuses them, turning quadratic work into linear work.

STRUCTURE YOUR PROMPT FOR CACHE HITS

System rules

Tool schemas

Few shot examples

stable prefix, cached

User question

changes every call, never cached

WHY PRACTITIONERS CARE



Cache size grows with sequence length, which is why long context eats GPU memory



Providers bill a cached prefix at a large discount and skip its prefill



A cache hit needs an exact prefix match, so put stable content first

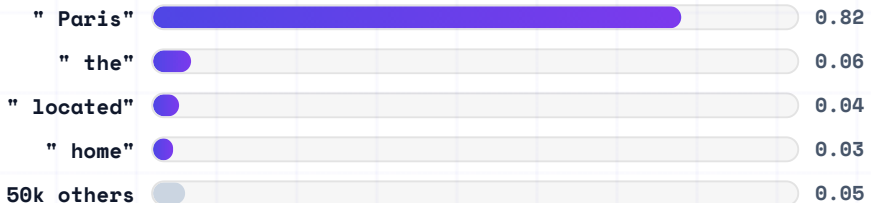
One dynamic timestamp at the top of your system prompt invalidates the entire cache on every request.

Logits to Probabilities

SOFTMAX

The final layer emits a raw logit for every token in the vocabulary. Softmax turns those into probabilities that sum to one.

PROMPT: "THE CAPITAL OF FRANCE IS"



LOOK AT IT YOURSELF

```
r = client.chat.completions.create(  
    model="gpt-4.1", messages=msgs,  
    logprobs=True, top_logprobs=5)  
for t in r.choices[0].logprobs.content[0].top_logprobs:  
    print(t.token, round(math.exp(t.logprob), 3))
```

READ THIS CAREFULLY



Every sampling knob reshapes this one distribution



None of them change what the model knows or believes



A confident shape can still be confidently wrong

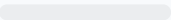
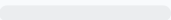
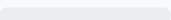
Temperature

SHARPNESS

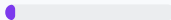
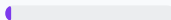
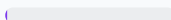
Temperature divides the logits before softmax. Low values sharpen the distribution toward the leading token. High values flatten it and give unlikely tokens a real chance of being drawn.

SAME LOGITS, THREE TEMPERATURES

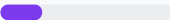
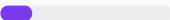
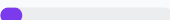
TEMP 0.0

Paris 
the 
located 
home 

TEMP 0.7

Paris 
the 
located 
home 

TEMP 1.5

Paris 
the 
located 
home 

WHERE TO SET IT



0 to 0.2 extraction and classification



0.2 code and structured output



0.7 chat and general writing



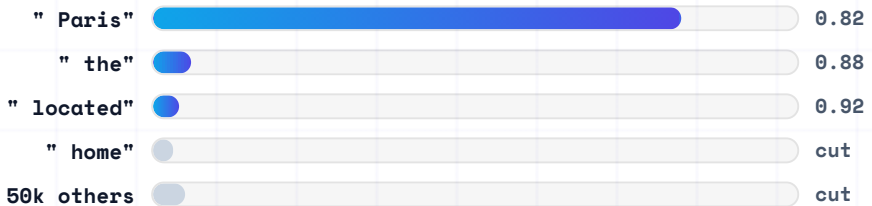
1.0 and up for idea variety

Temperature is a creativity dial only in the loosest sense. Past about 1.5 you are mostly buying incoherence.

Top p, Nucleus Sampling

TRUNCATION

Sort tokens by probability, add them up, and keep the smallest set whose total reaches p . Everything below the cut is discarded, then the survivors are renormalised and one is sampled.

TOP_P = 0.9, THE NUCLEUS HOLDS THREE TOKENS

The right column is the running total. It crosses 0.9 at the third token, so the pool closes there.

WHY IT BEATS A FIXED CUTOFF

↔ The pool shrinks to one token when the model is confident

🔲 It widens automatically when the model is genuinely unsure

🧹 It removes the long tail of nonsense that causes stray weirdness

Top k and Knob Order

STACKING

Top k keeps a fixed number of candidates regardless of the shape of the distribution. It is blunter than top p, but it is a hard guard against the long tail.

THE ORDER THESE ARE APPLIED

Logits › Penalties › Temperature › Top k › Top p › Sample

TOP_K = 40

- Always exactly 40 candidates
- Ignores the distribution shape
- Too wide when confident
- Too narrow when uncertain

TOP_P = 0.9

- Pool size adapts per token
- Follows the probability mass
- The better default of the two
- Pair with temperature, not top k

```
resp = client.messages.create(  
    model="claude-opus-4-5", messages=msgs,  
    max_tokens=1024,  
    temperature=0.7,      # how sharp the curve is  
    top_p=0.9,            # where the tail gets cut  
) # setting both at once is the classic mistake
```

Move one knob at a time. Tuning temperature and top p together makes any result impossible to attribute to either.

The Rest of the Knobs

CONTROL

Beyond temperature and top p, four parameters decide how repetitive, how long and how reproducible your output is.

PARAMETER	WHAT IT DOES
<code>frequency_penalty</code>	Penalises a token more each time it reappears. Kills word loops.
<code>presence_penalty</code>	One flat penalty after a token appears once. Pushes new topics.
<code>stop</code>	Strings that end generation immediately when produced.
<code>seed</code>	Makes sampling repeatable for the same input. Best effort only.

```
resp = client.chat.completions.create(  
    model="gpt-4.1", messages=msgs,  
    temperature=0.2,  
    max_tokens=512,  
    frequency_penalty=0.3, # stop it repeating itself  
    stop=["\n\nUser:"],   # never write the next turn  
    seed=42,              # reproducible in tests  
)
```

Penalties above 0.5 start damaging fluency, because common connecting words get punished too.

Why Generation Stopped

SIGNALS

Every response carries a stop reason. Ignoring it is how truncated answers quietly reach your users and your parsers.

STOP REASON	WHAT IT MEANS
<code>end_turn</code>	The model finished on its own. The only clean case.
<code>max_tokens</code>	You hit the ceiling. The answer is cut mid sentence.
<code>stop_sequence</code>	One of your stop strings was produced.
<code>tool_use</code>	The model is asking you to run a tool and come back.

STREAMING DOES NOT CHANGE ANY OF THIS

```
with client.messages.stream(**kwargs) as stream:
    for text in stream.text_stream:
        print(text, end="")
    final = stream.get_final_message()

if final.stop_reason == "max_tokens":
    handle_truncation(final)  # never just ship it
```

Streaming improves perceived latency, not real latency. The last token still arrives at the same moment it always would.

Roles Are Just Tokens

STRUCTURE

System, user and assistant look like separate fields in the API. Underneath, they are flattened into one token stream with special delimiter tokens the model was trained to respect.

<|system|>

You are a support agent

<|user|>

Where is my order

<|assistant|>

generation starts here

```
messages = [  
  {"role": "system",      "content": rules},  
  {"role": "user",        "content": "Where is my order?"},  
  {"role": "assistant",   "content": "Checking now."},  
  {"role": "user",        "content": "Any update?"},  
]  
# the full list is resent on every single call
```

CONSEQUENCES PEOPLE MISS



The model is stateless, your history is what creates memory



Turn ten costs far more than turn one, same model



Instructions in user text are just text, treat them as untrusted

Cutoffs and Hallucination

REALITY

Weights are frozen at a training cutoff, and the model has no way to tell a memorised fact from a plausible guess. Both come out of the same sampling process with the same confident tone.

WHY A WRONG ANSWER SOUNDS IDENTICAL TO A RIGHT ONE



Nothing after the cutoff exists unless you put it in context



Fluency is optimised directly, factuality only indirectly



Saying "I do not know" is a behaviour you have to ask for

THE STANDARD DEFENCE

Ground with
Retrieval

,

Constrain the
Output

,

Cite the
Source

,

Verify
Before Use

```
SYSTEM = ""Answer only from the context below.  
If the context does not contain the answer, reply  
exactly: I could not find this in the documents.  
Cite the source id after every claim.""
```

Temperature zero does not make a model truthful. It only makes it repeat the same answer, right or wrong, every time.

Settings That Actually Work

DEFAULTS

Start here, then change one parameter at a time and measure. These cover most production workloads.

TASK	TEMP	TOP_P	MAX_TOKENS
Classification, extraction	0.0	1.0	64
JSON and structured output	0.0	1.0	512
Code generation	0.2	0.95	2048
RAG answers with citations	0.1	0.9	1024
Chat assistant	0.7	0.9	1024
Brainstorming, variants	1.0	0.95	800

RULES THAT COME WITH THE TABLE

1 Set temperature or top p, leave the other at its default



Always set max_tokens, an unbounded cap is a runaway bill

The Whole Vocabulary



Tokens



Context window



Max output



Prefill



Decoding



KV cache



Logits and softmax



Temperature



Top p and top k



Stop reasons

Anyone can call the API.
Engineers know what every
parameter is doing.

Save this for your next LLM build. Follow for more.